Data-Structures & Algorithms

with

Manoj Prabhakaran

Lecture 36

Graphs

Spanning Trees

Spanning Arborescences

- A *spanning arborescence* of a directed graph G = (V,E) is a subgraph A=(V,E') (i.e., E' ⊆ E) which is a "directed tree": i.e., one node is a root with no incoming edges, while every other node is reachable from the root and has in-degree 1.
  - Number of edges is in E' exactly |V| - 1
- BFS, DFS, and Dijkstra all give spanning arborescences of the subgraph reachable from the source
  - For each reachable v, E' includes exactly one edge (u,v) such that v is discovered from u
    - If v discovered multiple times, one of them is selected: the first in BFS, the last in DFS, the one in a shortest path in Dijkstra

Minimum Spanning Arborescences

- In a weighted directed graph, Dijkstra gives a spanning arborescence A that minimises the total root-to-node distance in A: $\sum_{v:\text{ reachable}} d_A(s,v)$
- But consider building a network connecting the source to all the nodes, and we seek to minimise the total network's cost: $\sum_{(u,v)\in A} w(u,v)$
  - This is called the minimum spanning arborescence (MSA)
  - Example application: in Natural Language Processing, for generating syntax trees (nodes are words, edge weights generated by neural networks)
- There is an algorithm to find the MSA: Chu-Liu/Edmonds
  - We shall not see this algorithm
- Instead we consider the same problem in the setting of *undirected graphs*
  - Minimum Spanning Tree

Attendance Break!

Login using CSE LDAP id at

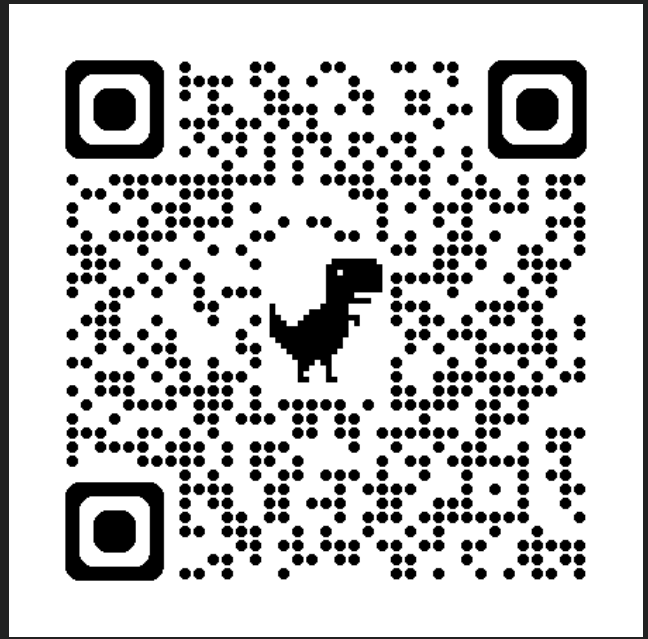
https://www.cse.iitb.ac.in/course_vm/cs213

Read the question, choose the answer and submit.
The points are mainly for attempting,
with a small bonus for getting it right.

Honour Code: Don't leave the lecture after marking attendance!

If you can't attend the whole lecture don't mark attendance!

If you are not in the classroom, a report will be sent to DADAC!



Undirected Graphs & Spanning Trees

- In an undirected graph, the edge set E is a set of sets of 2 elements (not ordered pairs). The edge is not an "arrow".
- A spanning tree of G=(V,E) is a subgraph T=(V,E') such that in T there is a *unique path* between any two nodes.
  - A tree is a connected graph with no cycles. Spanning tree touches all of V.
  - Tree to arborescence: Pick *any* node as the root. Direct all edges "away from the root" (along the unique path from root to one of the two nodes)
  - |E'| = |V|-1
- In a weighted undirected graph, weights on the undirected edges
  - A Minimum Spanning Tree is a spanning tree with minimum total weight

Finding MST

- Finding a minimum spanning tree has many applications:
  - Examples: Wiring in a VLSI circuit to minimise conductor length (total heat dissipated due to resistance). Cheapest power grid to connect an electricity substation to homes.
- We'll see two "greedy" algorithms, both selecting one edge at a time
  - Prim's algorithm: Grow a single tree
    - Like Dijkstra: uses a priority queue
  - Kruskal's algorithm: Grow many trees and connect them
    - Uses Union-Find to maintain a sets of nodes in each tree being grown
  - Both take $O(|E| \log |V|)$ time, but could be faster depending on how graph is given (e.g., as a list of edges sorted by weight)

Prim's Algorithm

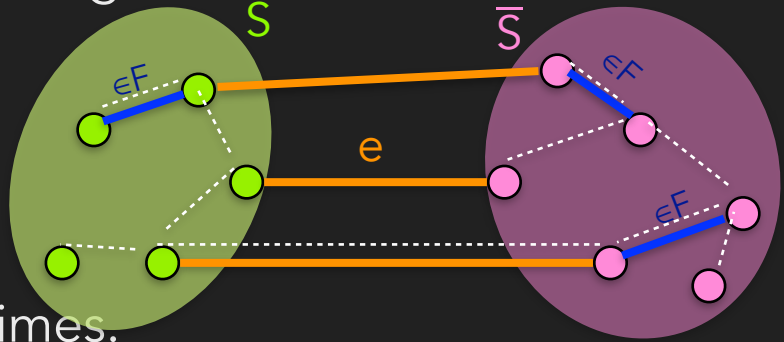
- Very similar to Dijkstra's algorithm: Grow a tree, incorporating one edge at a time (starting from an arbitrary source), selected using a priority-queue

```
Prim() {
  F = [];                                // F is a set of edges
  for (v:V) frontier.insert(v,⊥,∞); // store (v,u,wt({u,v})): v seen from u
  frontier.decrease(s,⊥,0);          // s is an arbitrary node
  while(!frontier.empty()) {
    (u,p) = frontier.extract_min(); // u seen from p is closest to F's nodes
    if(p != ⊥) F.add(p,u);
    for ( v: graph.neighbours(u) ) {
      wuv = graph.weight(u,v);
      frontier.decrease(v,u,wuv);   // update (v,x,j)→(v,u,wuv) iff wuv < j
    }
  }
  return F;
}
```

- Why does it work?

Underlying "Cut Observation"

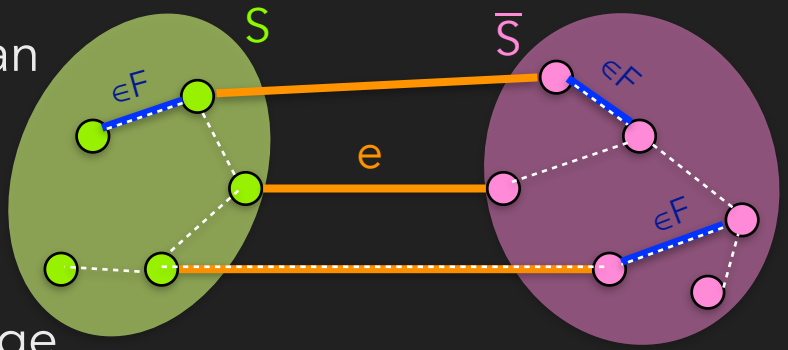
- Both algorithms will rely on the following observation for correctness
- For $S \subseteq V$, let $X_S = \{ \{u,v\} \in E \mid u \in S, v \notin S \}$ be the cut between S and $\overline{S}$
- Suppose $F \subseteq E'$ for some MST (V,E'), and e is a minimum weight edge in a cut X_S where $X_S \cap F = \emptyset$. Then there is an MST (V,E'') such that $F \cup \{e\} \subseteq E''$
- That is, it is *safe to add e to F* when constructing an MST, if e is a minimum weight edge in a cut that avoids F
  - Firstly, $E' \cap X_S \neq \emptyset$. If $e \in E'$ done (E''=E').
  - Else, $E' \cup \{e\}$ has a cycle through e (Why?)
  - A cycle crosses a cut an even number of times.
    So at least one more edge e' in the cycle. Take $E'' = E' \setminus \{e'\} \cup \{e\}$



connected graph with |V| edges: not a tree

Underlying "Cut Observation"

- It is *safe to add e to F* when constructing an MST, if e is a minimum weight edge in a cut that avoids F



- Take E'' = E' \ {e'} ∪ {e}, where e'≠e is an edge in the MST E' and is in the cut. e is a minimum weight edge in the same cut.
- Claim: (V,E'') is an MST
  - Total weight of E'' ≤ Total weight of E' (since wt(e) is min in the cut).
  - (V,E'') is connected (since the removed edge e' is from a cycle in E'∪{e}).
  - (V,E'') has no more cycles (since connected and |E''| = |E'| = |V|-1)
- F ∪ {e} ⊆ E'' (since F ⊆ E \ {e'}) ✓

Prim's Algorithm Revisited

- Maintain a tree (S,F), such that at any point there is an MST containing it
- Start with S={s} for an arbitrary node s and F = ∅
- Until S equals V, $X_S \neq \emptyset$ (some node in $\overline{S}$ has an edge to S) and $X_S \cap F = \emptyset$ (F is entirely within S)
  - At each iteration, pick a minimum weight edge $e \in X_S$.
  - Update S to $S \cup e$ (one of the end points is already in S) and F to $F \cup \{e\}$. By Cut Observation, safe to add e to F.
- When finished, S=V and, since we maintained the invariant, we have an MST!

Kruskal's Algorithm

- Maintain a *forest* (V,F) such that at any point there is an MST containing it
  - Forest: a graph with no cycles, but possibly not connected
- Start with F=∅. |V| trees, each with a single node.
- At each iteration, identify the minimum weight edge e which can be added between two trees in the forest.
- Continue until only one tree. This is an MST.
- Why? Suppose the edge identified is e, connecting two trees (A,T) and (A',T') in the forest. Every edge in X_A joins A with another tree in the forest. wt(e) is minimum among all such edges (and more). So by cut observation, F∪{e} is still contained in an MST.

Implementing Kruskal's Algorithm

- In each iteration, need the minimum weight edge that is between two trees
  - Go through the edges, in ascending order by weight (need to sort edges, in O(|E| log |E|) time unless the graph is already given with sorted weights)
  - For each edge {u,v}, we need to check if both u, v in the same tree
  - If not, we select the edge, and the two trees need to be united into a single tree
  - Union-Find! For each tree, maintain a set of all its nodes. Checking if {u,v} is within a tree or across two trees: Find. Adding {u,v} to the forest to combine two trees: Union.
    - O(|E| α(|V|)) $\approx$ O(|E|) time once sorting is done